

UML

MARIANA ACEVEDO GAÑAN

CEDULA:1035973358

BASES DE DATOS SQL Y NOSQL

FICHA:3169892

2025

Preguntas de Introducción:

- **¿Qué es un diagrama?**

Un diagrama es una forma gráfica de mostrar información organizada. Sirve para representar relaciones, procesos o estructuras, lo que permite entender con mayor facilidad temas complejos o grandes volúmenes de datos..

- **¿Qué tipos de diagramas conoce o ha utilizado?**

- Diagramas de **flujo**: Representan secuencias de pasos o decisiones dentro de un proceso.
- Diagramas de **Venn**: Muestran similitudes y diferencias entre conjuntos de datos o conceptos.
- Diagramas **UML**: Usados en desarrollo de software para modelar componentes del sistema (clases, casos de uso, actividades, etc.).
- Diagramas de **árbol**: Estructuran datos en forma jerárquica, útiles para decisiones, genealogías o estructuras organizativas

- **• ¿Cómo se estructura un diagrama?**

Aunque depende del tipo de diagrama, en general todos incluyen:

1. **Elementos representativos**: Formas básicas como cuadros, círculos o nodos que simbolizan datos, acciones o conceptos.
2. **Conectores**: Flechas o líneas que marcan relaciones, secuencias o dependencias entre los elementos.
3. **Texto explicativo**: Etiquetas que identifican o describen cada componente.
4. **Organización lógica**: Distribución de los elementos con un orden visual coherente.
5. **Claridad visual**: Debe ser limpio, sin exceso de información ni elementos decorativos innecesarios.

- **¿Cómo se construye un diagrama?**

Establecer el objetivo: Definir con exactitud qué se quiere representar.

Elegir el tipo de diagrama adecuado: Dependerá de si se trata de un proceso, estructura, cronograma, comparación, etc.

Identificar los componentes: Reunir todos los datos, acciones o conceptos que deben aparecer.

Definir relaciones: Determinar cómo se conectan entre sí los elementos.

Hacer un borrador: Crear una versión preliminar para probar la disposición.

Usar herramientas: Puede hacerse en papel o con software como Draw.io, Lucidchart, Visio, PowerPoint o similares.

Revisar y corregir: Verificar que todo sea comprensible, coherente y sin errores.

- **¿Cuál es el objetivo de usar diagramas?**

Mostrar información de forma visual, clara y ordenada. Esto permite analizar, comunicar y entender mejor los datos o estructuras que serían más complicados de explicar solo con texto.

- **¿Por qué consideras que deben usarse diagramas en el diseño de soluciones de software?**

- Permiten ver la estructura del sistema antes de programar.
- Ayudan a entender cómo interactúan usuarios, módulos y bases de datos.
- Sirven como una guía clara para todos los miembros del equipo.
- Reducen errores en la interpretación de requerimientos.
- Facilitan la detección de fallas o mejoras antes de codificar.
- Mejoran la comunicación entre desarrolladores, analistas y clientes.
- Hacen más sencillo mantener o modificar el software a futuro.

Investigue:

- **¿Cuáles son los tipos de diagrama más relevantes en UML? – Describa al menos 4 de ellos.**

UML (**Unified Modeling Language**) es un lenguaje de modelado utilizado en ingeniería de software para diseñar y visualizar sistemas antes de su implementación. Dentro de UML, existen varios tipos de diagramas esenciales

1. Diagrama de casos de uso

- Representa **interacciones** entre usuarios y el sistema.
- Muestra **actores** (usuarios, sistemas externos) y **casos de uso** (funcionalidades principales del sistema).
- Útil para definir **requerimientos funcionales** y alcance del sistema.
- Ejemplo: Un usuario que inicia sesión en una aplicación y accede a su perfil.

2. Diagrama de clases

- Modela la **estructura de un sistema orientado a objetos**.
- Define **clases**, atributos, métodos y relaciones entre ellas.

- Es clave para diseñar la **arquitectura de software**, especialmente en lenguajes como Java o C++.
- Ejemplo: Una clase "Usuario" con atributos como "nombre" y "email", y métodos como "registrar()" y "actualizarDatos()".

3. Diagrama de secuencia

- Representa el **orden en el que ocurren las interacciones** entre objetos.
- Muestra mensajes intercambiados en **líneas de tiempo**, útil para visualizar flujos de procesos.
- Facilita la comprensión de lógica interna en módulos de software.
- Ejemplo: El flujo de autenticación en una aplicación con validación de contraseña.

4. Diagrama de actividad

- Describe **procesos secuenciales** o flujos de trabajo dentro del sistema.
- Usa **acciones, decisiones y bifurcaciones** para representar estados y transiciones.
- Ayuda en el modelado de **algoritmos y flujos de negocio**.
- Ejemplo: El proceso de compra en un comercio electrónico desde la selección de productos hasta el pago.

- **¿Cómo representar instancias de las clases en UML? -Elabore un ejemplo de una clase. Con sus atributos y características.**

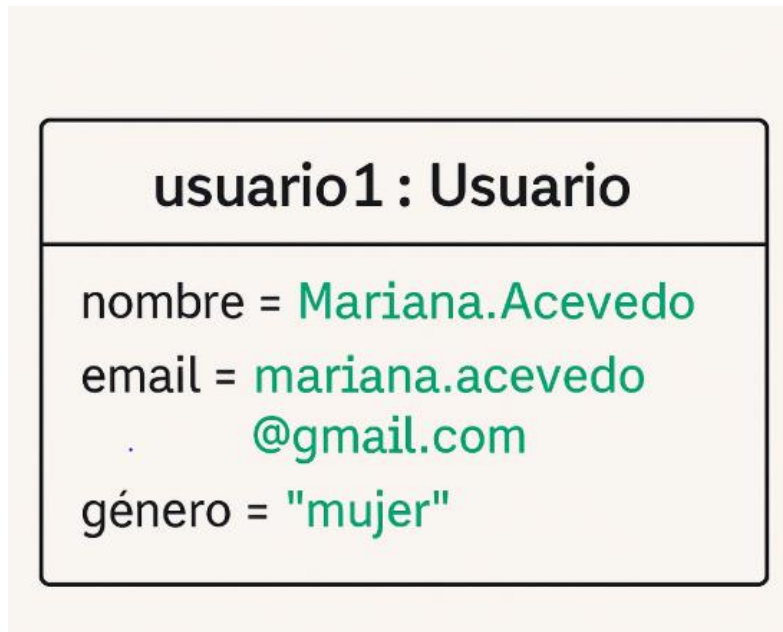
En UML, una **instancia de clase** se representa con un **diagrama de objetos**. Este tipo de diagrama muestra objetos concretos derivados de clases, incluyendo los valores específicos que contienen. Sirve para ver cómo se comportan las clases cuando trabajan con datos reales dentro del sistema.

Clase:usuario

nombre: string

email:string

edad: int



- ¿ UML permite incluir notas, cómo, para qué?

Si, **UML permite incluir notas o comentarios** en los diagramas, y son muy útiles para **aclarar información, agregar explicaciones o dar contexto adicional** que no forma parte del modelo estructural o de comportamiento.

¿Cómo se representan las notas en UML?

- Las **notas** se dibujan como un **rectángulo con la esquina superior derecha doblada**, como si fuera una hoja doblada.
- Se pueden **conectar con una línea punteada** al elemento (clase, objeto, relación, etc.) al que se refieren.

¿Para qué sirven las notas en UML?

Por ejemplo, explicar por qué una clase tiene ciertos atributos o por qué se toma una decisión de diseño.

1. **Documentar restricciones o reglas:** Como condiciones que no se pueden modelar gráficamente directamente, por ejemplo: "La edad no puede ser negativa".
2. **Anotar decisiones técnicas o pendientes:** Por ejemplo: "Este componente aún está en revisión" o "Se planea reemplazar esta clase en la siguiente versión".
3. **Facilitar la comunicación entre el equipo:** Las notas permiten que otros desarrolladores, diseñadores o clientes entiendan mejor el propósito de ciertos elementos del diagrama.

- **¿Qué es un método constructor y para qué se utiliza?**

Un **método constructor** es una función especial que se utiliza en programación orientada a objetos para **crear e inicializar un objeto** a partir de una clase. Su función principal es **asignar valores iniciales** a los atributos del objeto cuando se crea una instancia.

- **¿Para qué se utiliza?**

- Para **asignar valores por defecto o personalizados** a los atributos del objeto.
- Para **asegurar que el objeto se cree con datos válidos** desde el inicio.
- Para ejecutar **acciones necesarias al momento de crear el objeto**, como conectar a una base de datos o cargar datos.

- **¿Qué tipos de Relaciones pueden especificarse entre clases?**

En UML, entre las clases se pueden especificar varios **tipos de relaciones** que representan cómo interactúan o se conectan entre sí. Las más comunes son:

- **¿Qué tipos de Relaciones pueden especificarse entre clases?**

En UML, las **relaciones entre clases** son fundamentales para definir la estructura y el comportamiento del sistema. Aquí están los tipos más comunes:

1 Asociación

- Representa una **relación estructural** entre dos clases.
- Se indica con una **línea** entre las clases y puede tener cardinalidad (1, 0..*, etc.).
- Ejemplo: Un `Cliente` puede estar asociado con varios `Pedidos`.

2 Agregación

- Es un tipo de asociación, pero con una **relación de "todo-parte"**.
- La clase agregadora **contiene** objetos de la otra clase, pero estos pueden existir de forma independiente.
- Se indica con un **rombo vacío** en la clase principal.
- Ejemplo: Un `Departamento` tiene varios `Empleados`, pero los empleados pueden existir sin el departamento.

3 Composición

- Similar a la agregación, pero con una **relación de dependencia fuerte**.

- Si el objeto principal se elimina, sus partes también lo hacen.
- Se indica con un **rombo lleno** en la clase principal.
- Ejemplo: Un `Auto` tiene `Motor`; si el auto se destruye, el motor también.

4 Herencia (Generalización)

- Representa una relación **padre-hijo**.
- Una subclase **hereda** atributos y métodos de la superclase.
- Se indica con una **línea con triángulo** apuntando a la clase padre.
- Ejemplo: `Estudiante` hereda de `Persona`.

5 Dependencia

- Indica que una clase **usa** a otra de manera temporal.
- Se representa con una **línea discontinua con flecha**.
- Ejemplo: `Usuario` depende de `ServicioAutenticación` para iniciar sesión.

Actividad 1: Diagrama de Casos de Uso en UML

¿Qué es un Diagrama de Casos de Uso (Use Case Diagram)?

Es un tipo de diagrama de UML (Lenguaje Unificado de Modelado) que **describe las funciones del sistema desde el punto de vista del usuario**. Ayuda a **entender qué hace el sistema** sin entrar en detalles técnicos.

Elementos que componen un Diagrama de Casos de Uso

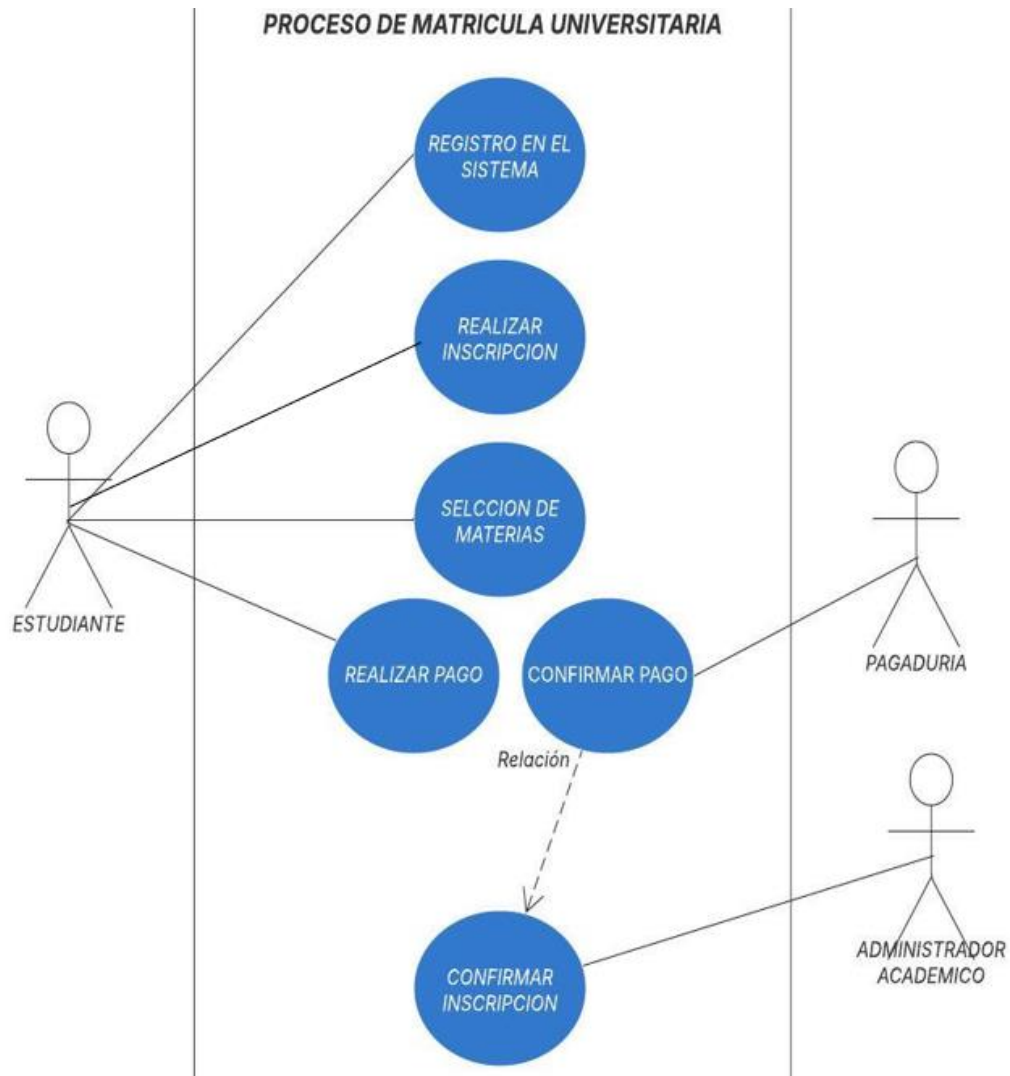
Elemento	Símbolo	Descripción
Actor	(muñeco o stickman)	Representa una persona, sistema externo o rol que interactúa con el sistema.
Caso de uso	● ó □ (óvalo)	Describe una función o acción que realiza el sistema en beneficio del actor.
Sistema	Rectángulo grande	Representa el límite del sistema donde ocurren los casos de uso.
Relaciones	Flechas y líneas	Conectan actores con casos de uso o casos de uso entre sí.

Tipos de Relaciones

1. **Asociación (actor ↔ caso de uso)**
Flecha o línea simple que indica que el actor **usa ese caso de uso**.
 2. **Include (<<include>>)**
Un caso de uso **siempre incluye** otro. Ejemplo: "Comprar producto" incluye "Validar pago".
 3. **Extend (<<extend>>)**
Un caso de uso **puede extender** otro en ciertas condiciones. Ejemplo: "Registrar usuario" puede extenderse con "Enviar correo de confirmación".
 4. **Generalización**
Un actor o caso de uso **hereda** comportamiento de otro. (Ej: "Administrador" hereda de "Usuario").
-

¿Cómo se crea paso a paso?

1. **Define los actores:**
¿Quiénes interactúan con el sistema? (ej. cliente, empleado, administrador)
2. **Identifica los casos de uso:**
¿Qué acciones puede hacer cada actor? (ej. iniciar sesión, pagar, buscar producto)
3. **Dibuja el sistema** como un rectángulo.
4. **Coloca los actores afuera** y los casos de uso **dentro del sistema**.
5. **Conecta** cada actor con los casos que usa.
6. Agrega relaciones <<include>> o <<extend>> si hay comportamiento compartido u opcional



Actividad 2: Diagrama de Clases en UML

¿Qué es un Diagrama de Clases en UML?

Un **diagrama de clases** muestra la **estructura estática** de un sistema. Representa las **clases**, sus **atributos**, **métodos (funciones)** y las **relaciones entre ellas**.

Elementos de un Diagrama de Clases UML

Elemento	¿Qué representa?
Clase	Una entidad del sistema. Ej: Usuario, Ahorro, Préstamo.
Atributos	Características de la clase. Ej: nombre, fecha, saldo.
Métodos	Acciones o funciones que realiza la clase.
Relaciones	Cómo se conectan entre clases (asociación, herencia...).

Análisis de la Actividad 1 (la natillera en Tokio)

Extraemos verbos, sustantivos y entidades:

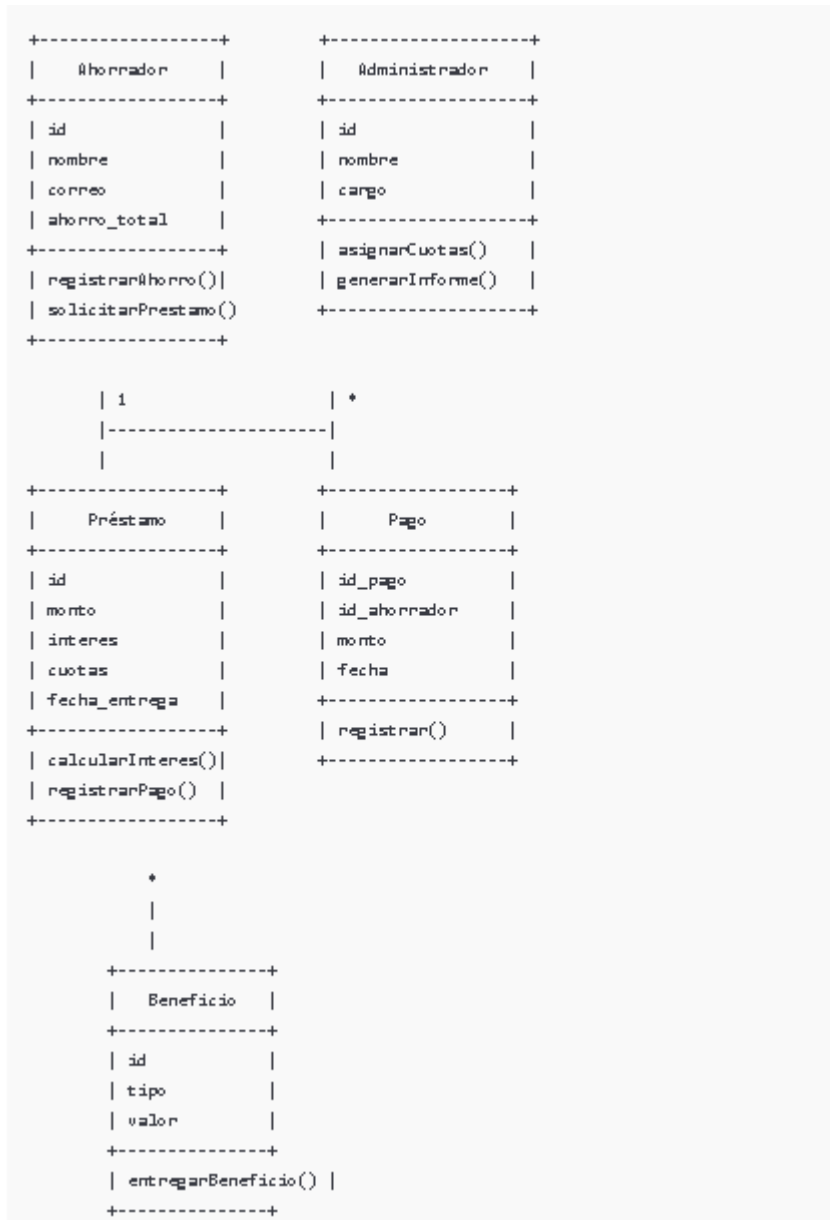
Tipo	Palabras encontradas
Sustantivos	ahorrador, administrador, prestamista, inversor, ciclo, cuota, préstamo, pago, beneficio, rifas, saldo, interés
Verbos	ahorrar, invertir, prestar, pagar, liquidar, obtener, entregar
Entidades (Clases)	Ahorrador, Administrador, Préstamo, Inversor, Cuota, Ciclo, Ahorro, Beneficio, Pago

Tabla de Clases con atributos y métodos sugeridos

Clase	Atributos	Métodos
Ahorrador	id, nombre, correo, ahorro_total	registrarAhorro(), solicitarPrestamo()
Administrador	id, nombre, cargo	asignarCuotas(), generarInforme()
Inversor	id, nombre, monto_invertido	invertir(), retirarInversión()
Préstamo	id, monto, interes, cuotas, fecha_entrega	calcularInteres(), registrarPago()
CicloCuota	id, fecha_inicio, fecha_fin, valor_cuota	crearCiclo(), cerrarCiclo()

Clase	Atributos	Métodos
Pago	id_pago, id_ahorrador, monto, fecha	registrar()
Beneficio	id, tipo (viaje, premio), valor	entregarBeneficio()

Diagrama de clases UML (versión textual simplificada)



GLOSARIO

- **UML (Unified Modeling Language):**
Lenguaje estándar de modelado visual utilizado para diseñar, visualizar y documentar sistemas de software orientados a objetos mediante diagramas como clases, casos de uso, secuencia, entre otros.
- **POO (Programación Orientada a Objetos):**
Paradigma de programación que organiza el software en objetos que interactúan entre sí. Cada objeto pertenece a una clase y tiene atributos (datos) y métodos (funciones).
- **Objeto:**
Instancia de una clase. Representa una entidad concreta con atributos definidos y comportamientos (métodos) específicos.
- **Clase:**
Plantilla o molde que define los atributos (propiedades) y métodos (acciones) comunes a un conjunto de objetos. Es el componente básico en la POO.
- **Modelo:**
Representación simplificada y visual de un sistema o parte de él. En UML, los modelos permiten comprender la estructura y comportamiento del software antes de implementarlo.
- **Atributo:**
Propiedad o característica de una clase u objeto. Por ejemplo, en una clase `Persona`, los atributos pueden ser `nombre`, `edad` o `correo`.
- **Alcance (o visibilidad):**
Nivel de acceso de un atributo o método dentro de una clase. Los más comunes son:
 - `public (+)`: accesible desde cualquier parte.
 - `private (-)`: accesible solo dentro de la clase.
 - `protected (#)`: accesible desde la clase y sus subclases.
- **Parámetro:**
Dato que se pasa a una función o método para que pueda operar con diferentes valores en diferentes ejecuciones.
- **Agregación:**
Relación entre clases donde una clase contiene a otra, pero sin dependencia total. Representa una relación “tiene un” (por ejemplo, un `Curso` tiene varios `Estudiantes`).
- **Herencia:**
Mecanismo mediante el cual una clase hija hereda atributos y métodos de una clase padre. Permite la reutilización de código y creación de jerarquías.
- **Polimorfismo:**
Capacidad de los objetos de responder de diferentes formas al mismo mensaje o método. Puede lograrse mediante **sobreescritura** o **sobrecarga**.

- **Extensión:**
En UML, se refiere a la relación entre un caso de uso principal y uno opcional que **extiende** su comportamiento bajo ciertas condiciones.
- **Función (o método):**
Acción que puede realizar un objeto. Es parte del comportamiento de una clase, y puede modificar o consultar sus atributos.
- **Relación:**
Conexión entre clases, objetos o actores en un modelo UML. Ejemplos: asociación, agregación, herencia, composición.
- **Abstracción:**
Proceso de ocultar los detalles internos y mostrar solo las funcionalidades esenciales. En POO, se logra creando clases generales que representan entidades conceptuales.
- **Sobrecarga:**
Definir varios métodos con el mismo nombre pero con distintos parámetros (cantidad o tipo). Permite que una función tenga múltiples formas de ser usada.